Grado en Ingeniería Informática

Tecnologías de Desarrollo de Software

Patrones de Diseño

(Parte 2. Patrones Estructurales)

Facultad de Informática

UNIVERSIDAD DE MURCIA

Departamento Informática y Sistemas

Contenidos

- Introducción a los patrones de diseño
- Patrones de **creación**
  - Factoría Abstracta, *Builder*, Método Factoría, Singleton
- Patrones **estructurales**
  - Adaptador, Composite, Decorador, Fachada
- Patrones de **comportamiento**
  - Command, Iterador, Observer, Estado, Estrategia, Método Plantilla

Patrones Estructurales

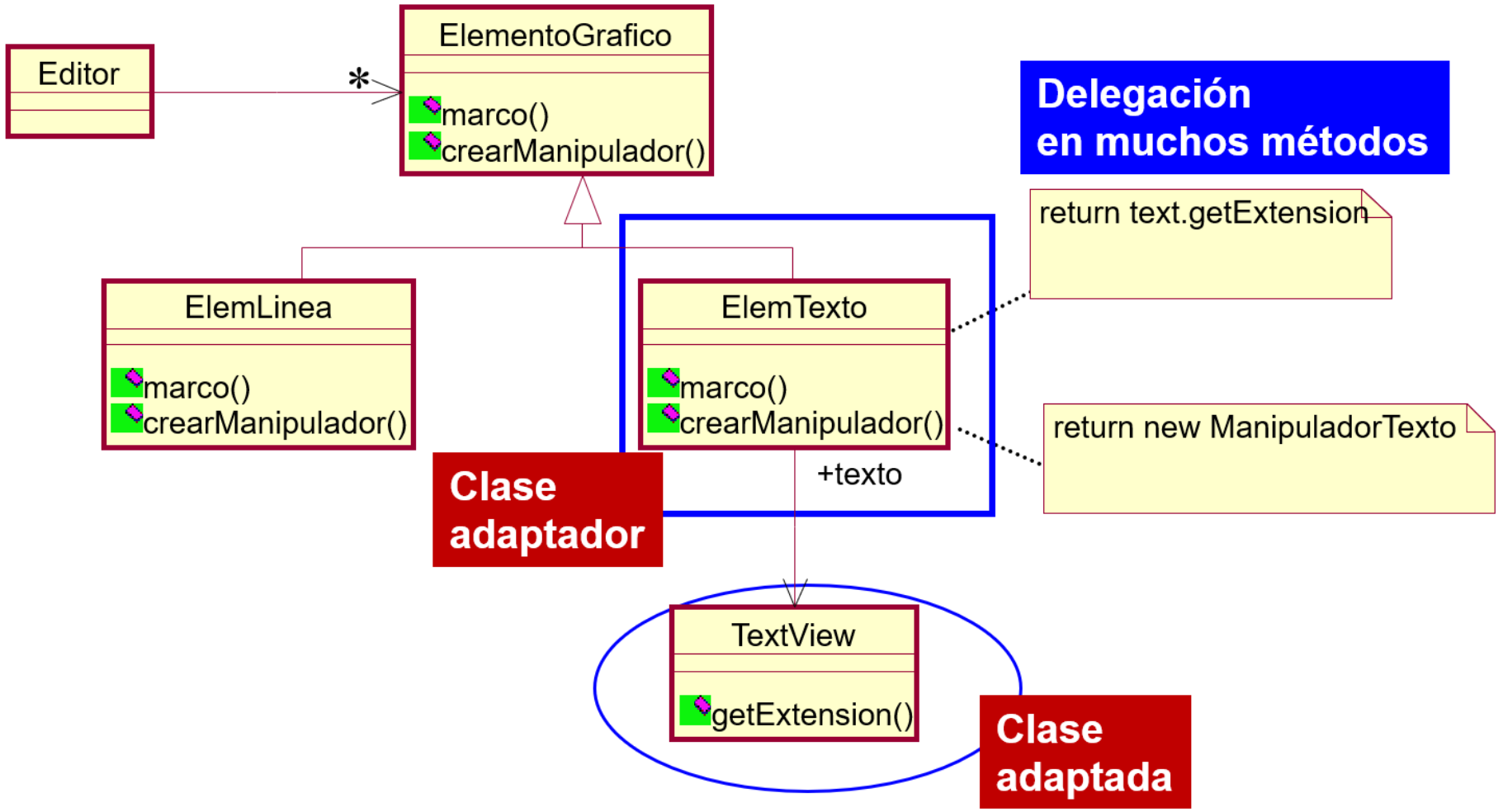
- Describen formas de combinar objetos para obtener una funcionalidad que resuelve problemas recurrentes en programación orientada a objetos.
  - Basadas en la **composición**.
  - Posibilidad de cambiar la estructura en tiempo de ejecución.

Patrón Adapter / Wrapper (Adaptador)

- **Propósito**
  - Convertir la interfaz de una clase en otra que una clase cliente espera. Permite la colaboración de ciertas clases a pesar de tener interfaces incompatibles

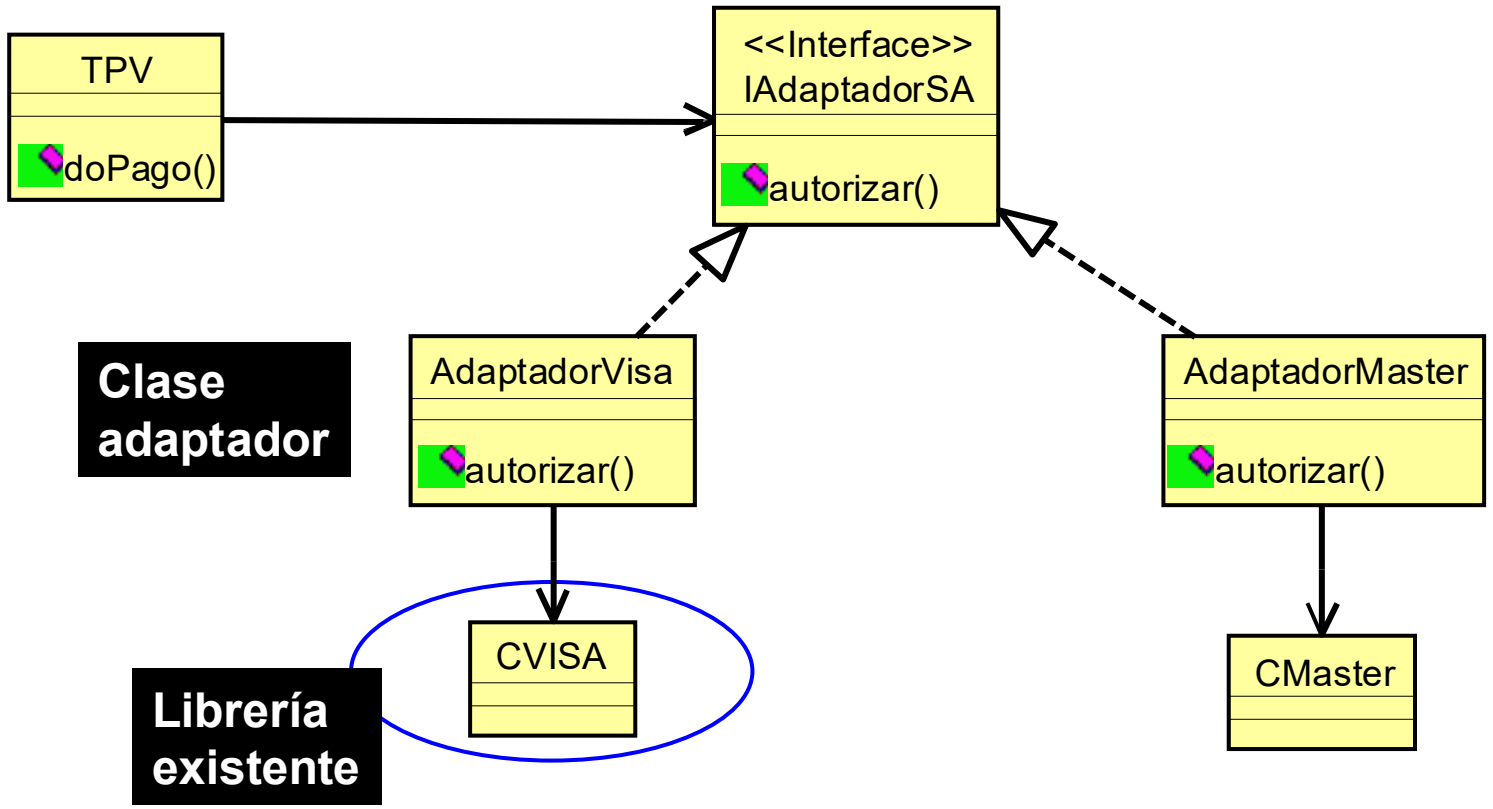
Patrón Adaptador: Motivación “Reuso”

Un editor gráfico incluye una jerarquía que modela elementos gráficos (líneas, polígonos, texto,..) y se desea reutilizar una clase existente `TextView` para implementar la clase que representa elementos de texto, pero que tiene una interfaz incompatible con la raíz de la jerarquía.

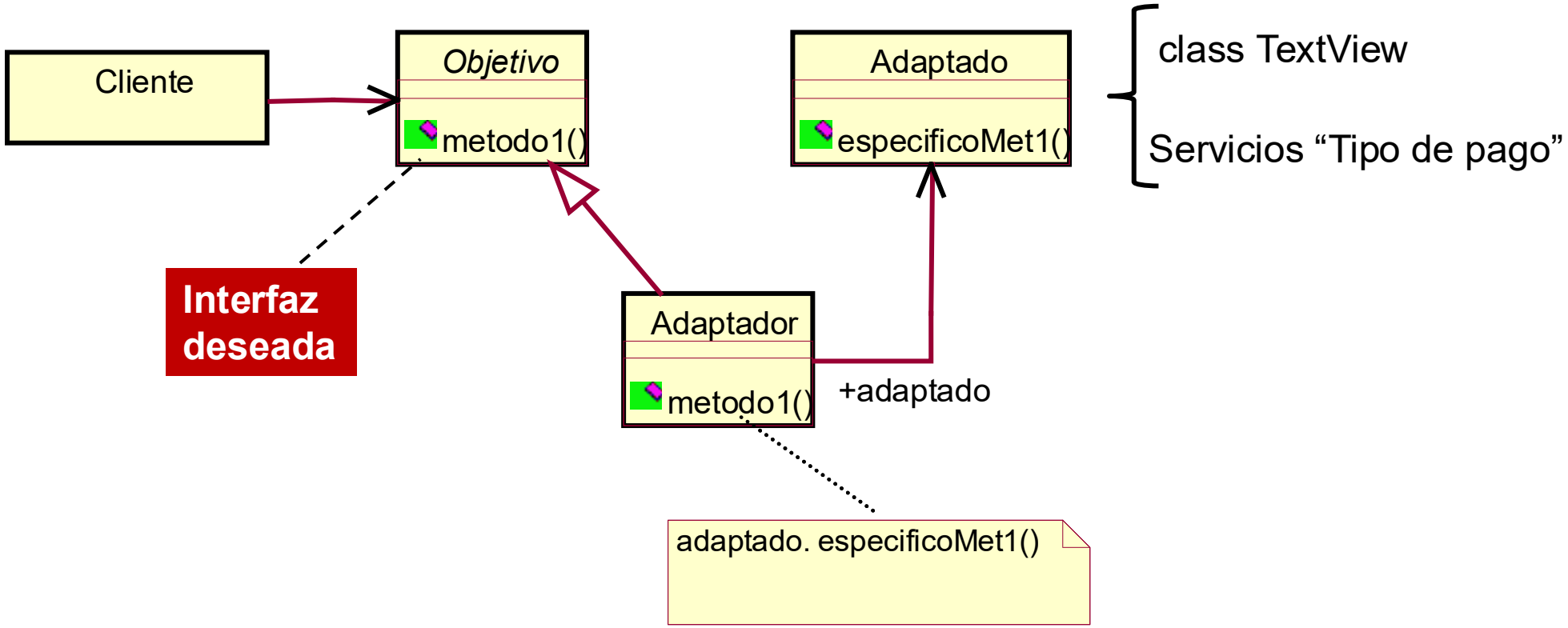


Patrón Adaptador: Motivacion "Variación en tipos"

En una aplicación de TPV se deben soportar diferentes tipos de servicios externos de realización de pagos o de inventario y se quiere ser **independiente de un servicio concreto.**



Adaptador: Estructura



Adaptador

Aplicabilidad

- Se desea usar una **clase existente y su interfaz no coincide con la que se necesita.**
- Se desea **crear una clase reutilizable** que debe colaborar con clases no relacionadas o imprevistas **(punto de variación).**

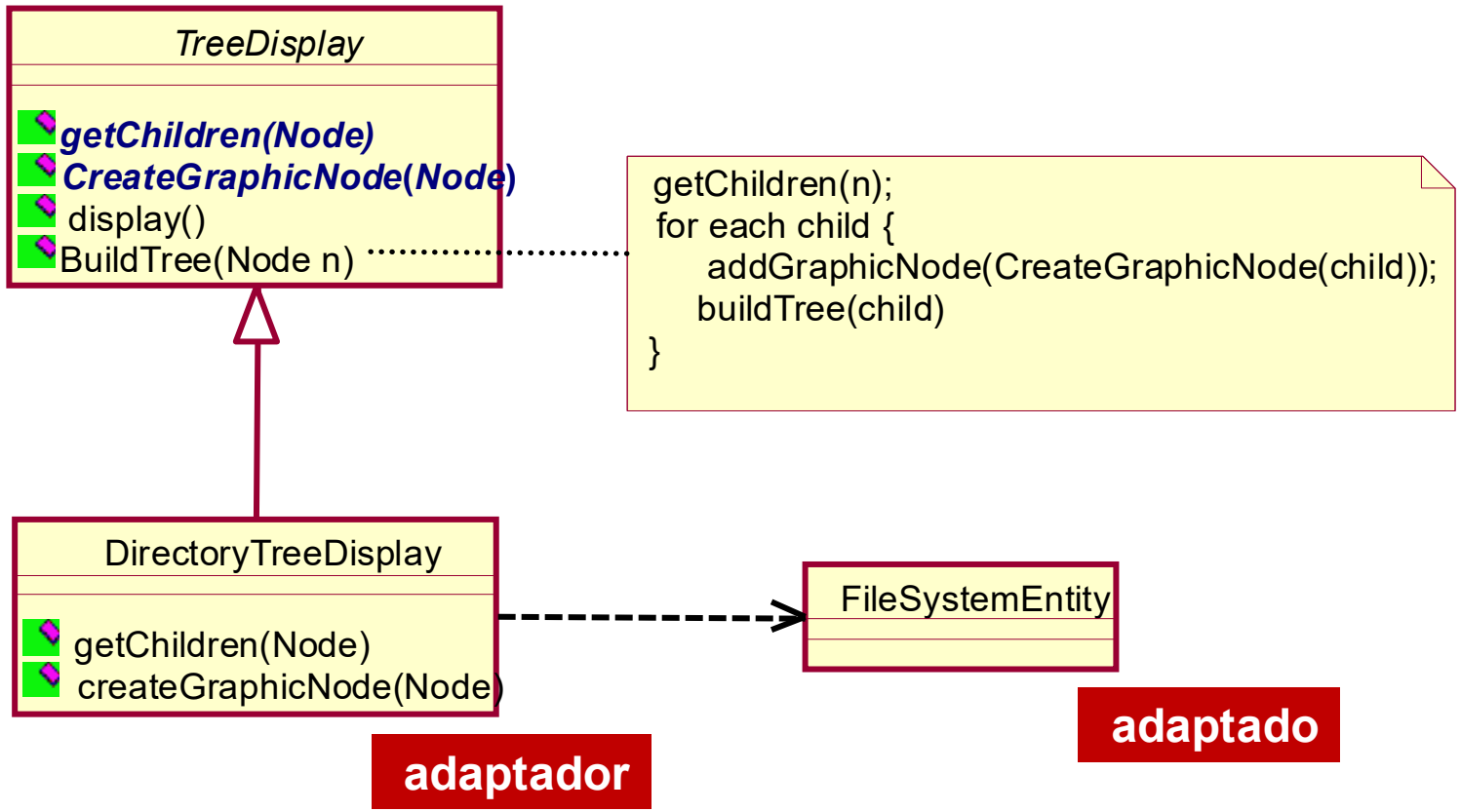
Consecuencias

El tamaño de la clase *Adaptador* depende de la similitud entre la interfaz de las clases *Objetivo* y *Adaptado*.

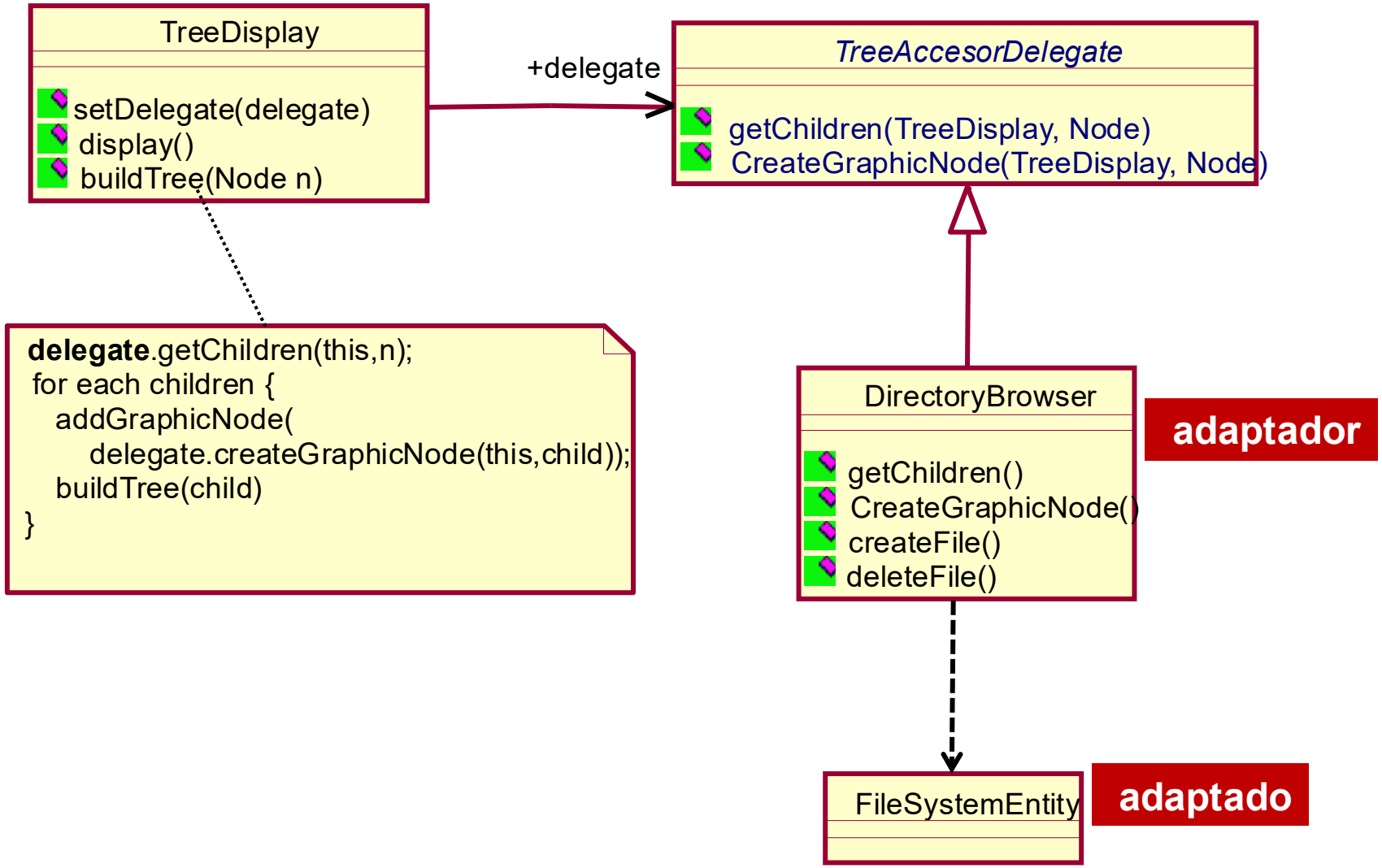
Ejemplo en libro GoF

Cómo crear una clase **`TreeDisplay`** para visualizar estructuras jerárquicas de cualquier naturaleza.

TreeDisplay es una clase abstracta y los adaptadores son subclases



TreeDisplay es una clase concreta que delega en una interfaz que implementan los adaptadores



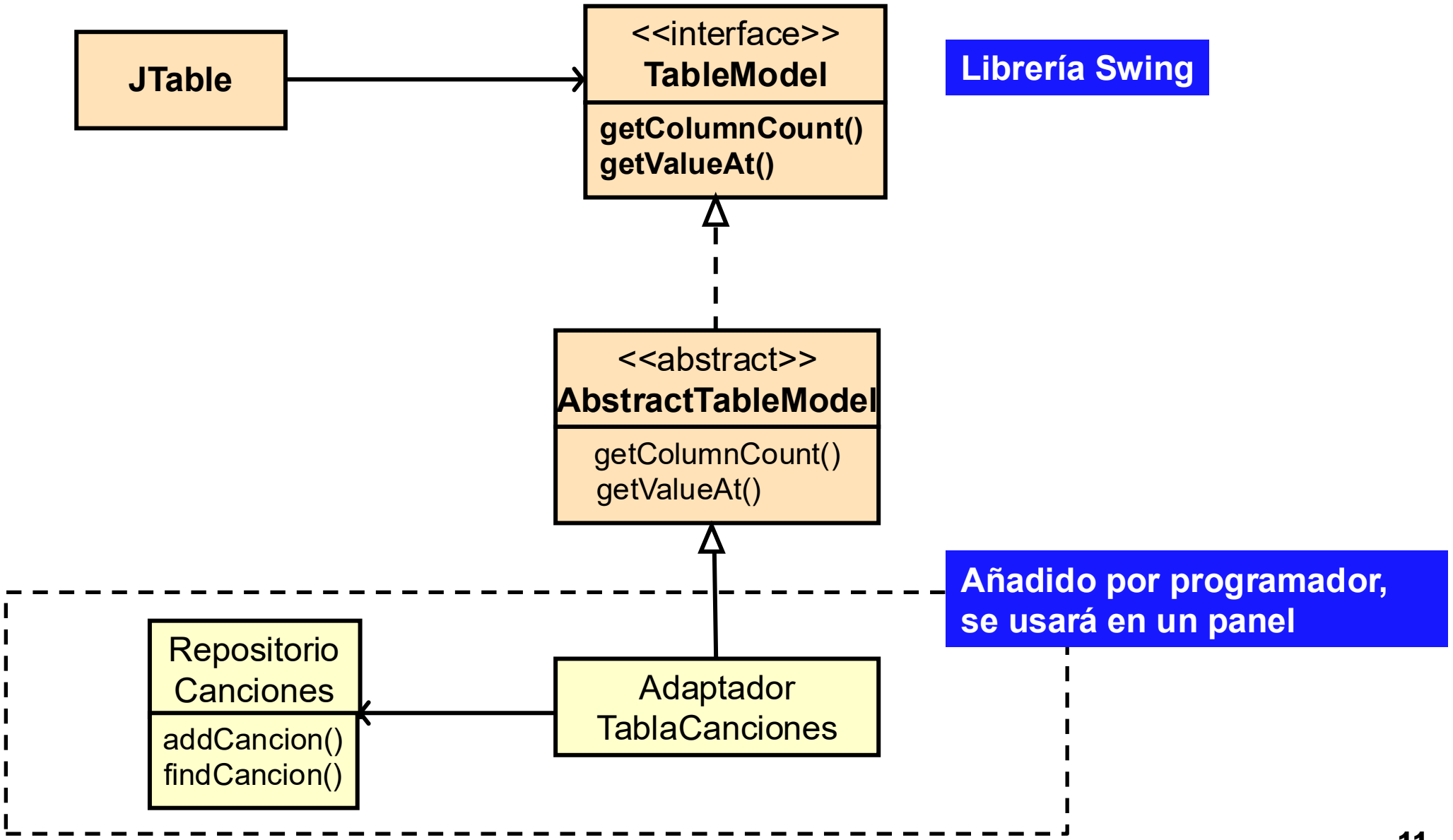
Adaptador *en Swing*

La librería Swing de Java incluye clases gráficas que muestran información organizada en cierto formato, como son `JTree` (información jerárquica), `JTable` (tabla bidimensional) o `JList` (lista de datos). Estas clases son clientes respectivamente de **`TreeModel`**, **`TableModel`** y **`ListModel`** que le aíslan de los detalles de un modelo de datos concreto y que incluyen métodos que le permiten obtener los datos a representar según el formato que corresponda (Similar a *Treedisplay* basado en composición.




```
public interface TableModel {
     // obtener número de filas
     public int getRowCount();
     // obtener número de columnas
     public int getColumnCount();
     // obtener valor del elemento fila fil y columna
     col
     public Object getValueAt(int fil, int col);
     // obtener nombre de la columna col
     public String getColumnName(int col);
}
```

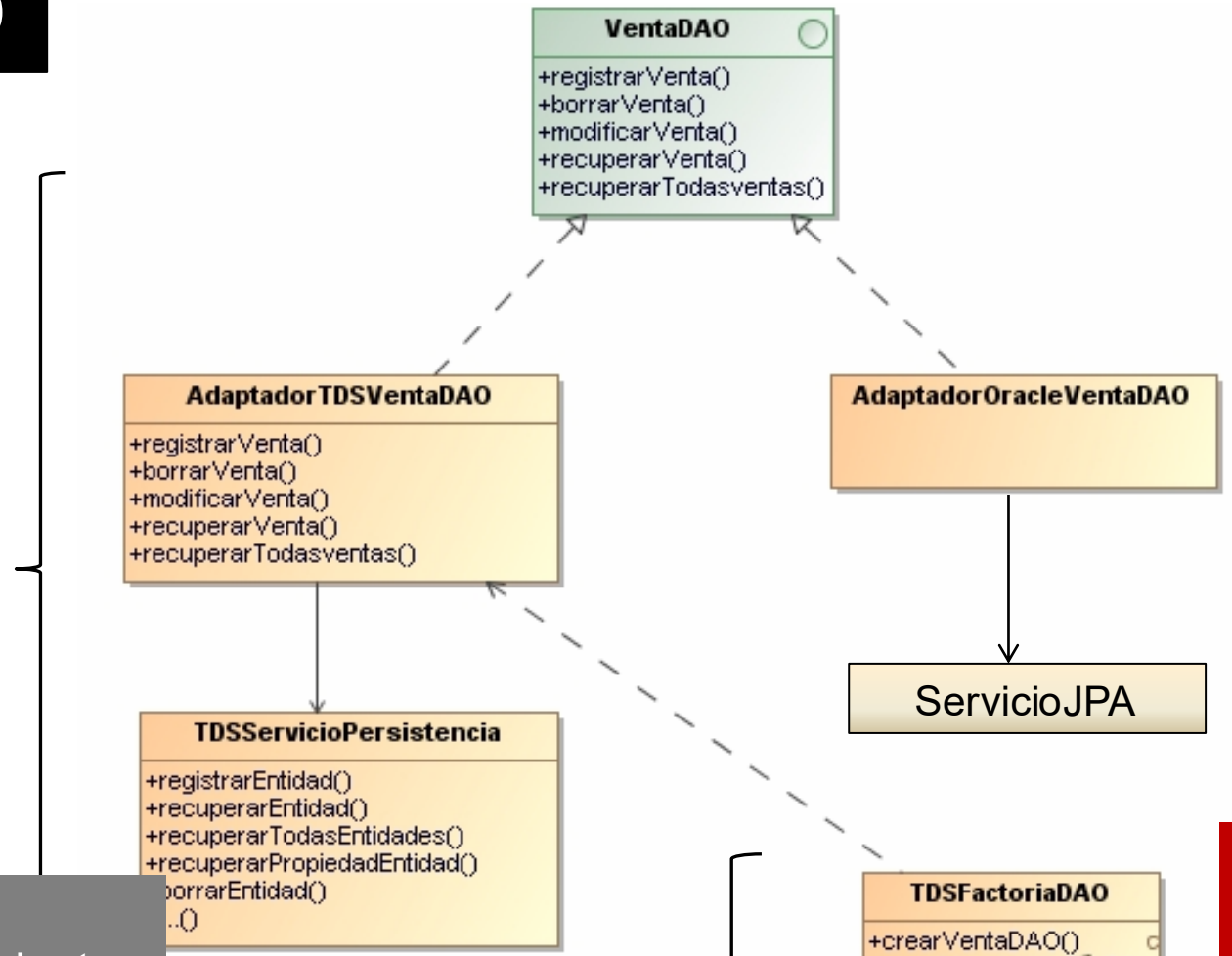
Adaptador *en Swing: JTable*



Patrón DAO: Adaptador + Factoría Abstracta + Singleton

- El patrón DAO (*Data Access Object*) se utiliza para conseguir que una aplicación sea independiente del sistema de almacenamiento utilizado (p.e., un sistema relacional de uno u otro fabricante, o JDBC, o una base de datos MongoDB, etc.) .
- Para cada clase del negocio (por ejemplo, Cuenta y Cliente) se crea una interfaz DAO que proporciona los métodos CRUD para crear, recuperar, modificar y eliminar datos almacenados.
- La interfaz DAO es implementada por clases que se encargan de establecer las conexiones a una fuente de datos concreta y encapsular cómo se accede a dicha fuente.
- El patrón DAO utiliza una **factoría abstracta** para crear las instancias de las clases DAO que requiere la aplicación, para conseguir que está sea independiente de un sistema concreto de almacenamiento.

Ejemplo Patrón DAO



Patrón *Adaptador*

Familia -> Servicios de almacenamiento

Producto -> objetos DAOs

```
VentaDAO crearVentaDAO() {
   return new AdaptadorTDSVentaDAO();
}
```

Patrón *Factoría Abstracta*

Patrón *Singleton*

Composite

Propósito

- Componer objetos en estructuras jerárquicas para representar jerarquías "*parte/todo*". Permite al código cliente manejar a los objetos **primitivos** y **compuestos** de forma uniforme.

Motivación

- **Figuras compuestas:** una figura compuesta es una figura formada por figuras primitivas y compuestas
- **Contenedores GUI**: un contenedor es un componente GUI formado por widgets y contenedores
- **Circuitos compuestos:** un circuito compuesto es un circuito formado por circuitos primitivos y compuestos.
- **Paquetes de valores:** un paquete es un valor formado por valores simples (acciones, bonos,..) y por paquetes.
- Un **menú** está formado por ítems y menús.

Figuras simples

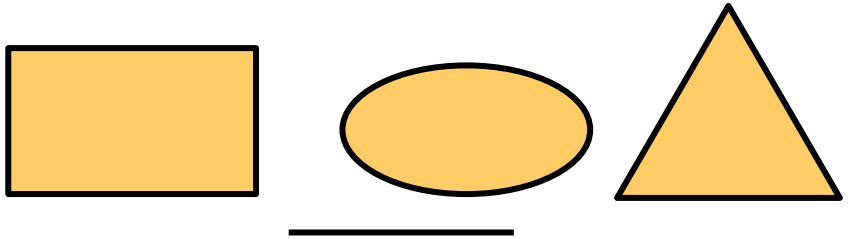
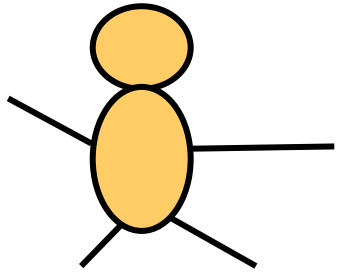


Figura compuesta



También es una una figura

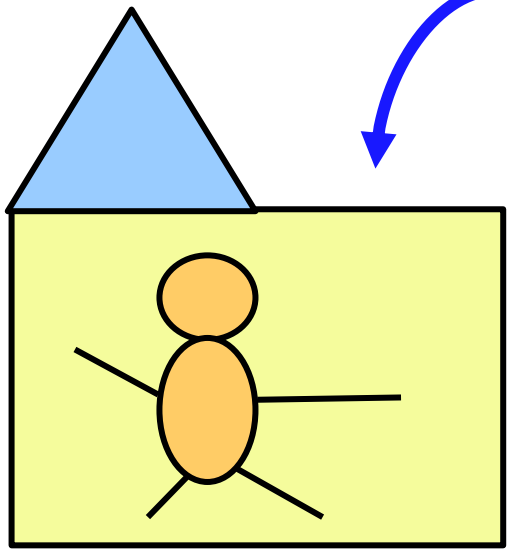
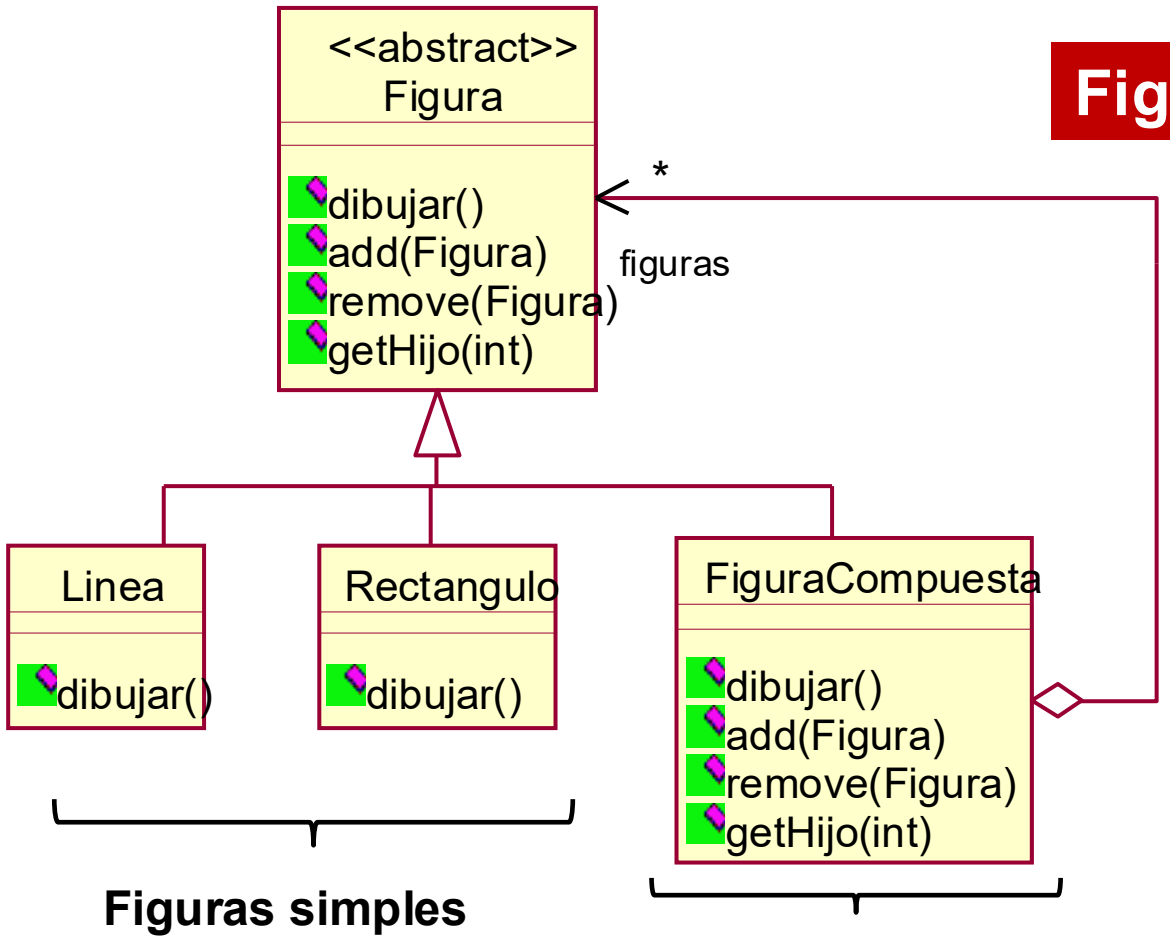


Figura compuesta

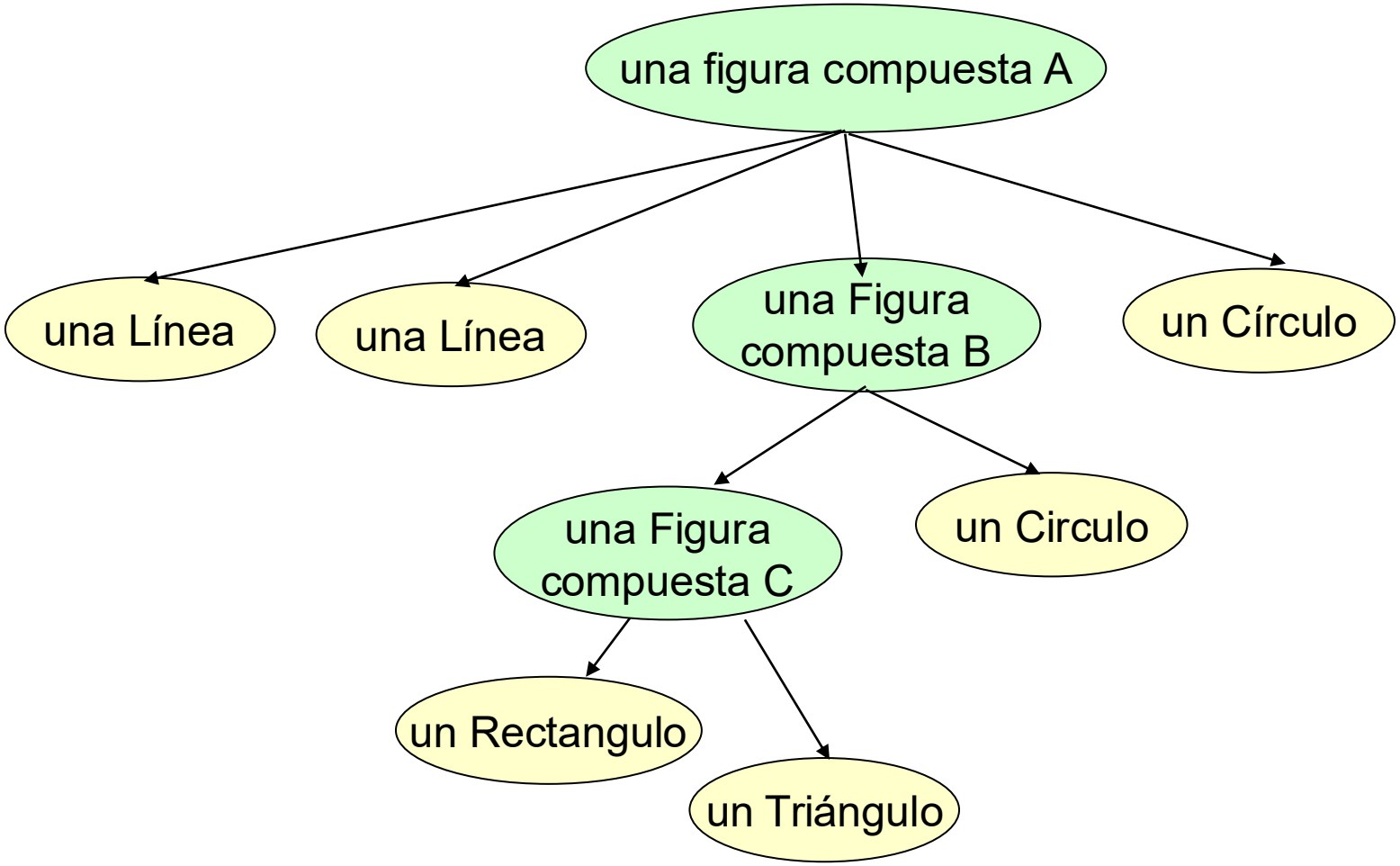
Composite: Motivación (1)



Figuras compuestas

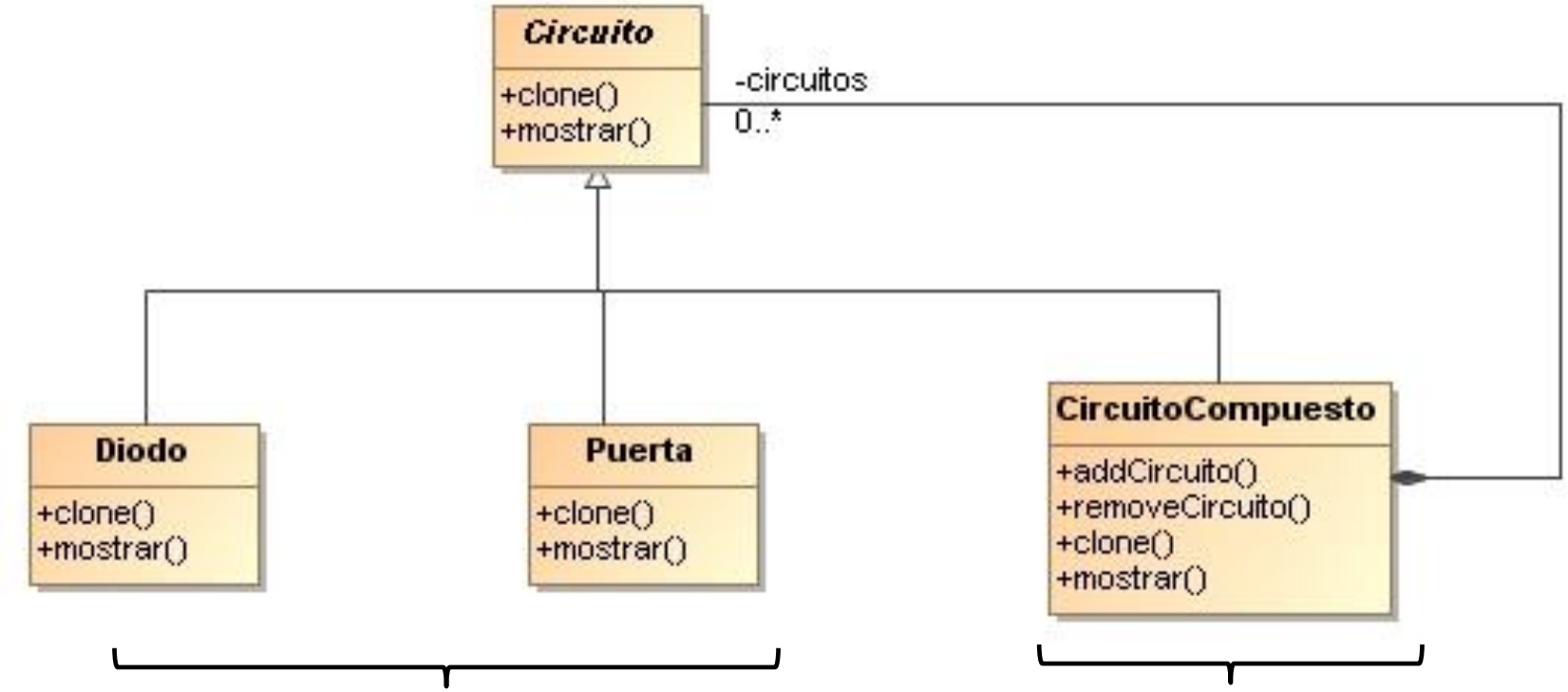
Composite: Motivación (1)

Composición recursiva



Composite: Motivación (2)

Circuitos compuestos

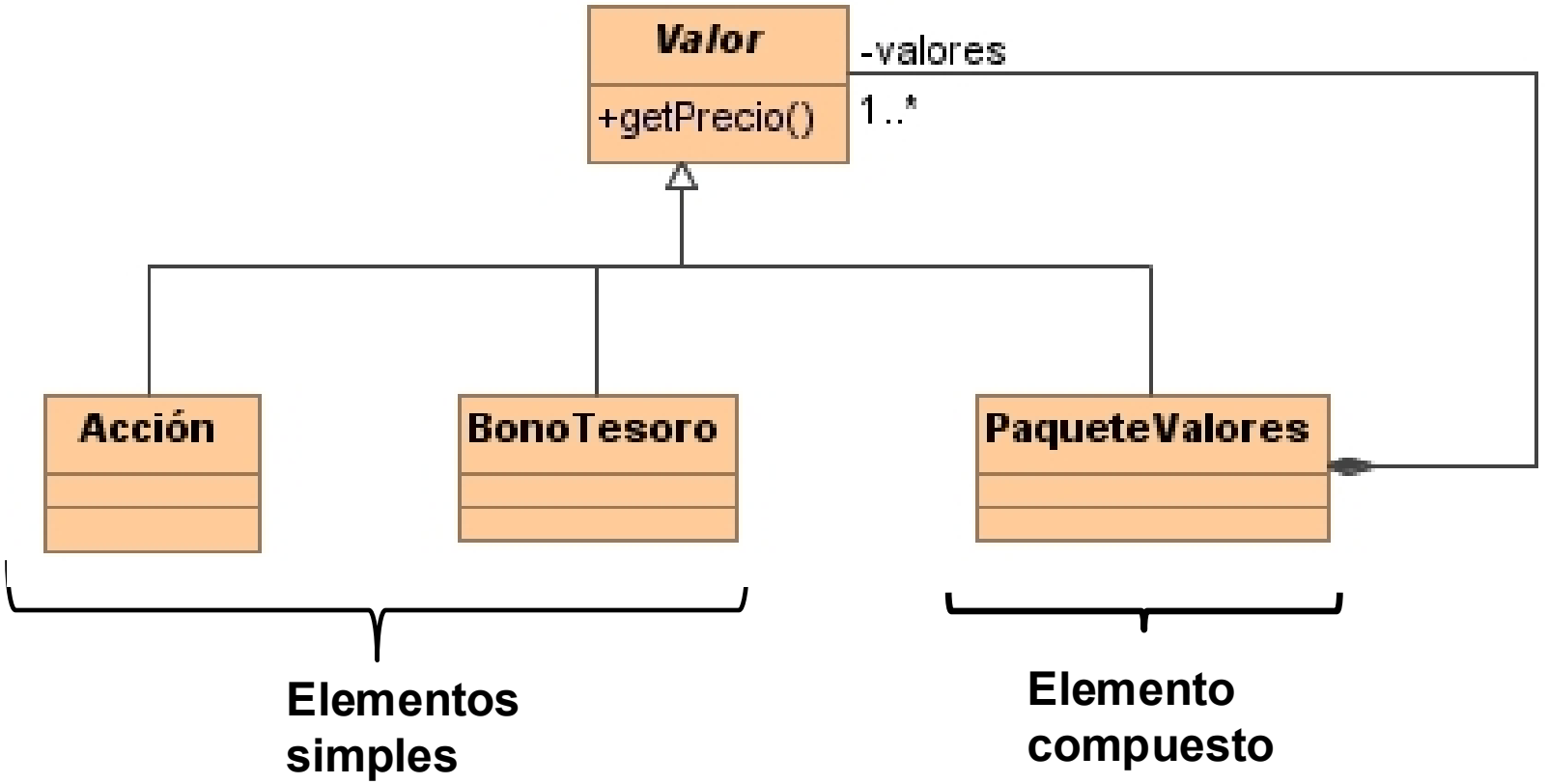


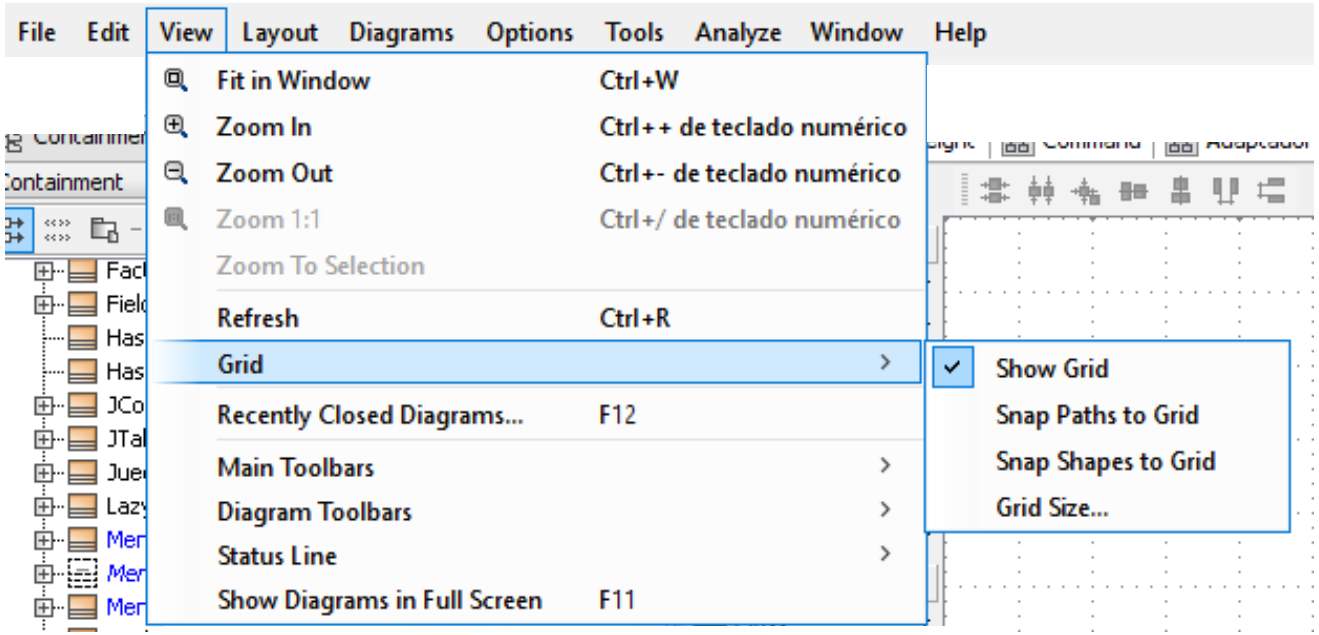
Circuitos simples

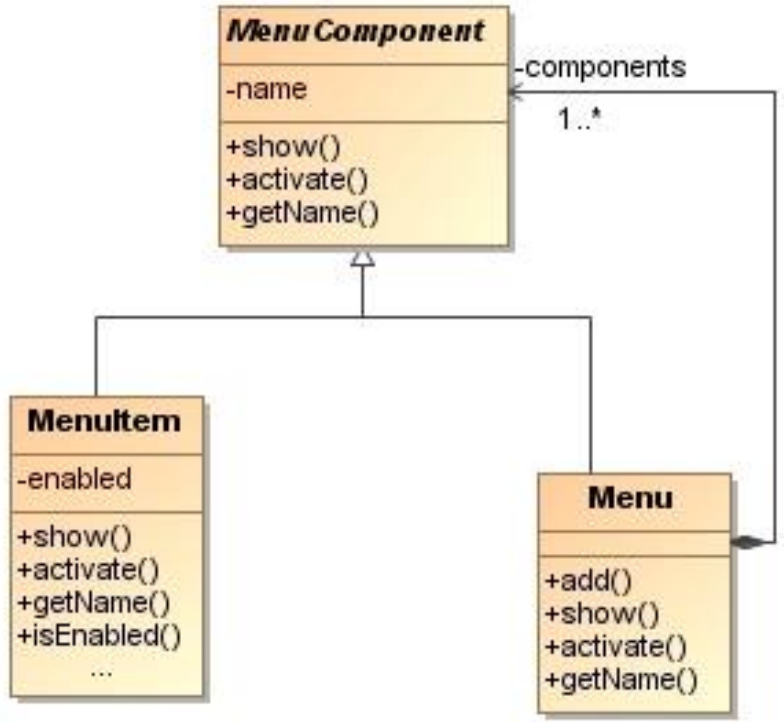
Circuito compuesto

Composite: Motivación (3)

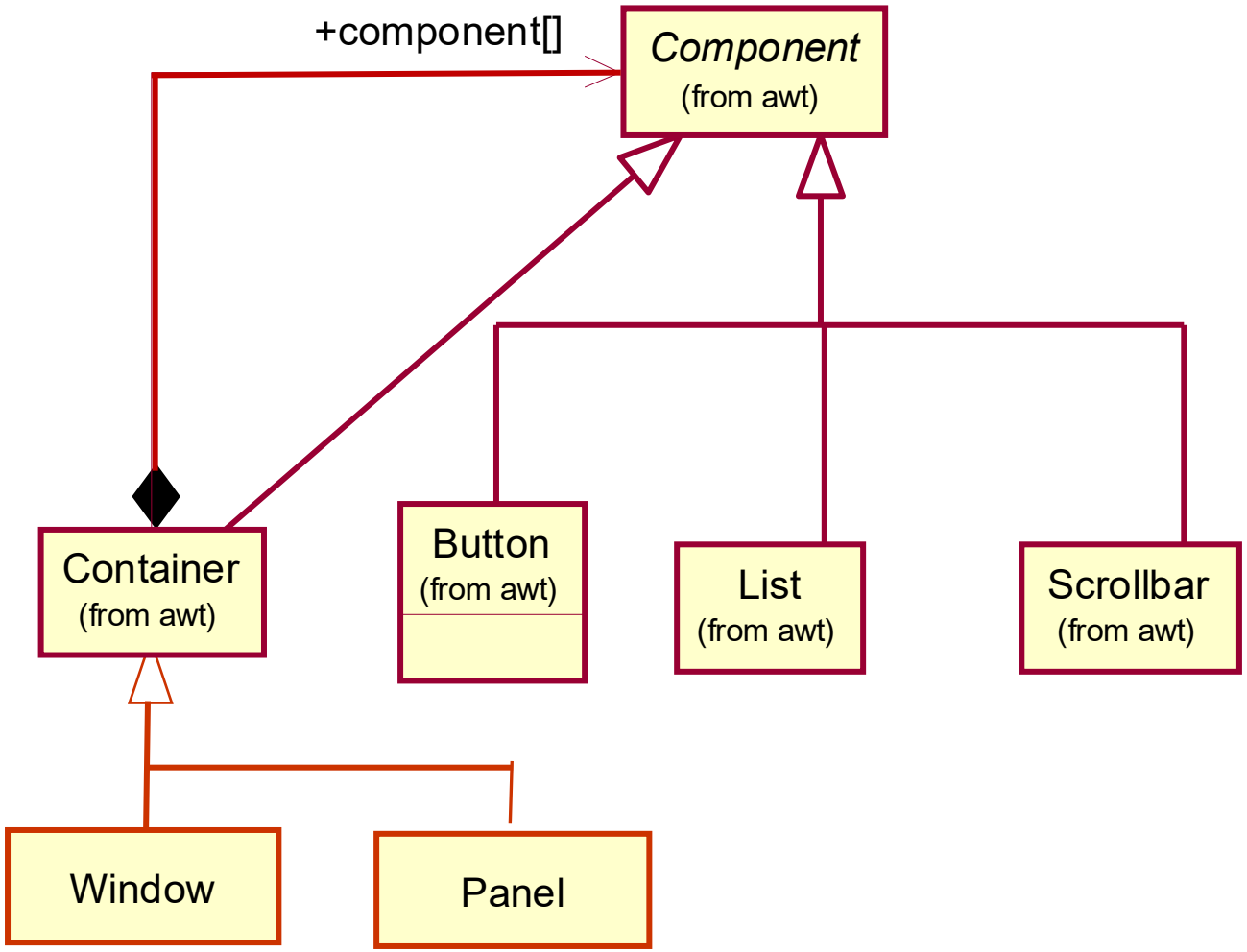
Paquetes de valores




File
Edit
View
Layout
Diagrams
Options
Tools
Analyze
Window
Help
Fit in Window Ctrl+W
Zoom In Ctrl++ de teclado numérico
Zoom Out Ctrl+- de teclado numérico
Zoom 1:1 Ctrl+/ de teclado numérico
Zoom To Selection
Refresh Ctrl+R
Grid
Recently Closed Diagrams... F12
Main Toolbars
Diagram Toolbars
Status Line
Show Diagrams in Full Screen F11
Show Grid
Snap Paths to Grid
Snap Shapes to Grid
Grid Size...
Containment


MenuComponent
-name
+show()
+activate()
+getName()
-components
1..*
MenuItem
-enabled
+show()
+activate()
+getName()
+isEnabled()
...
Menu
+add()
+show()
+activate()
+getName()

Component GUI: Ejemplo en Java AWT



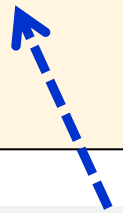
Composite: Ejemplo en Java AWT

Role
Composite

```
public class Container extends Component {..
    public Component add (Component c) {..}
    public void remove (Component c) {..}

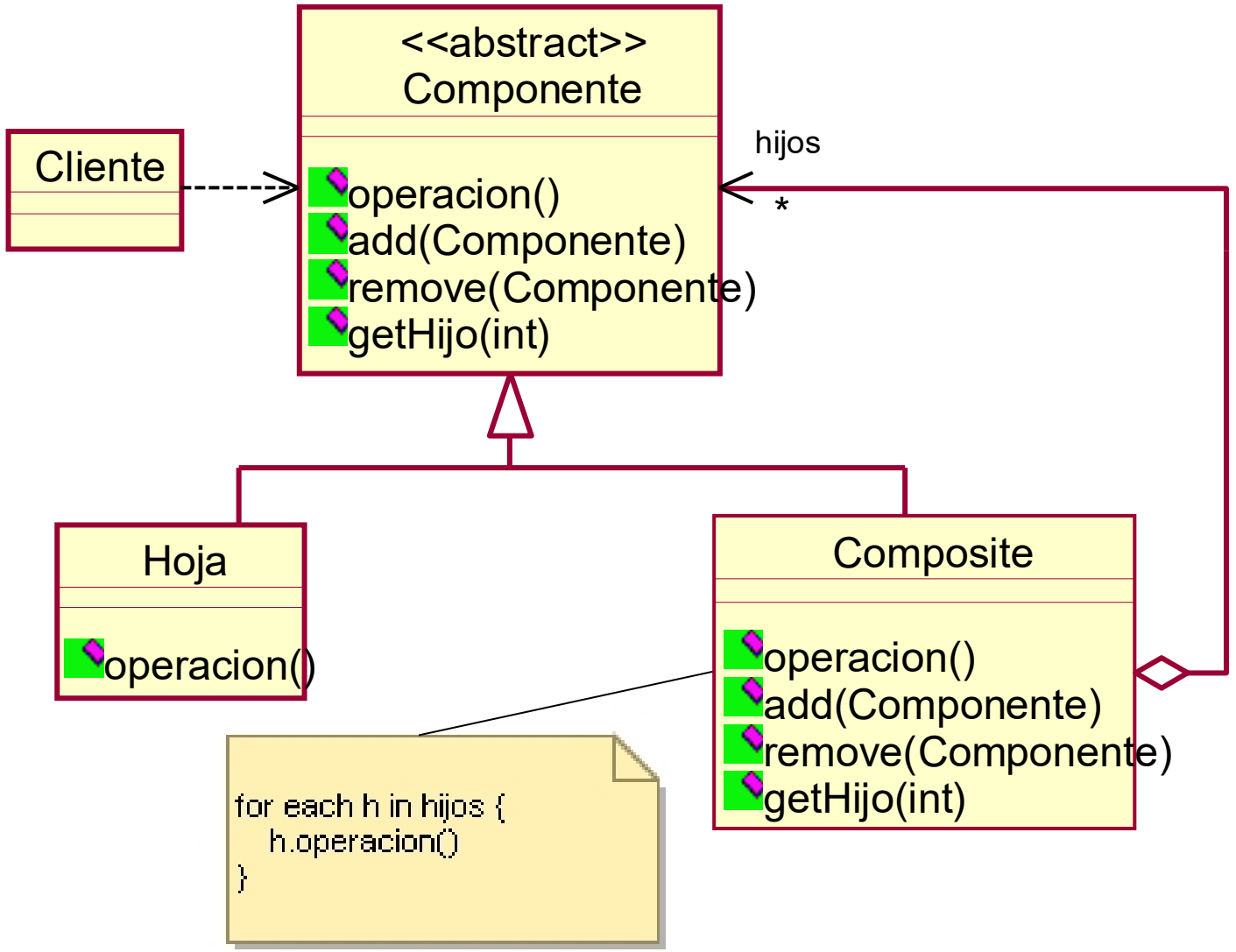
}
```

```
public class Window extends Container {
   Component[] components;
   public void update() {
        if (components != null)
              for (int k = 0; k < components.length; k++)
                                    components[k].update();
   }
}
```

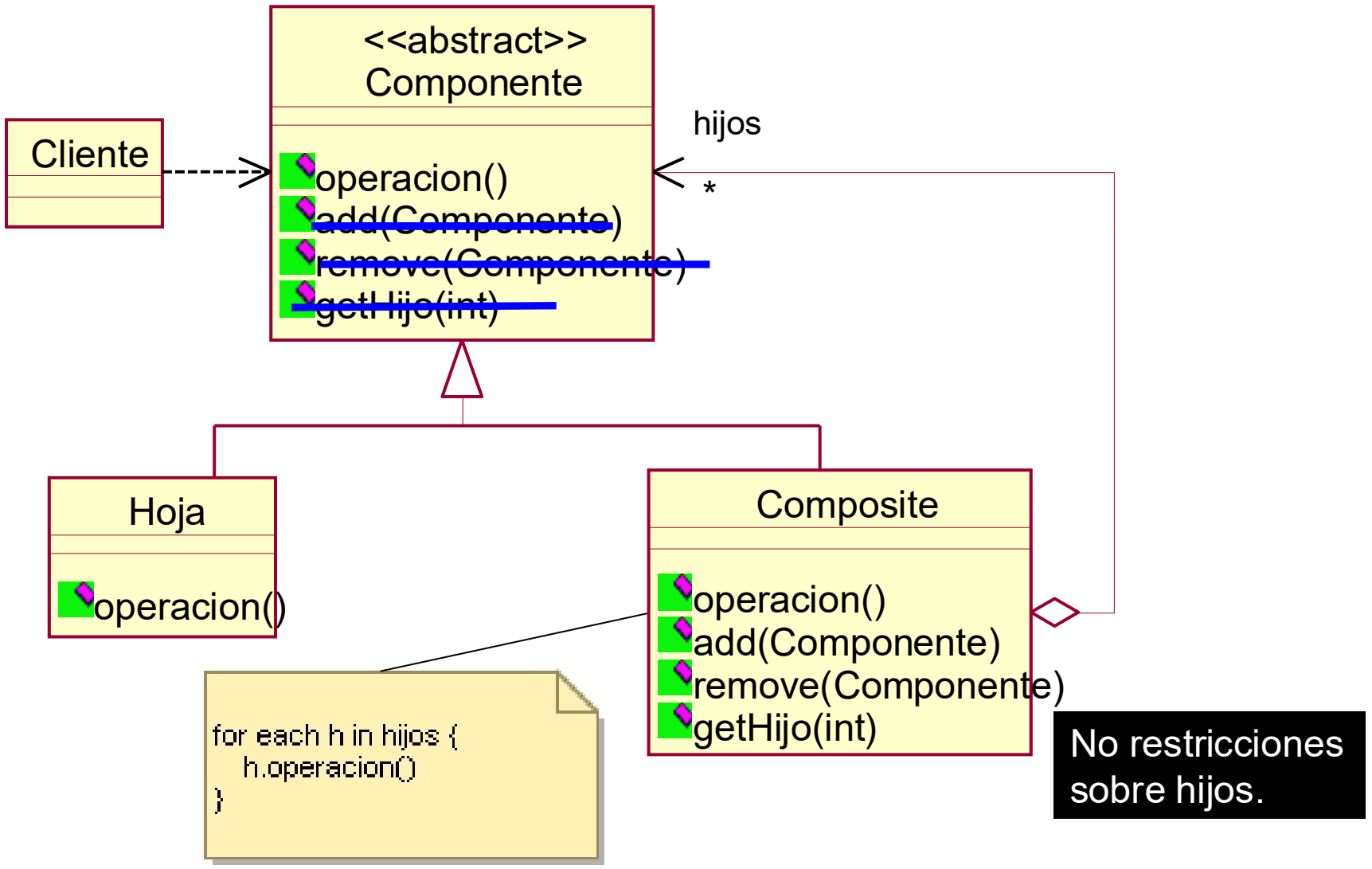


En la clase *composite* las operaciones heredadas normalmente se implementan recorriendo los elementos contenidos y aplicando la operación sobre cada elemento.

Composite: Estructura GoF



Composite: Estructura



Composite

Aplicabilidad

- Se quiere representar **jerarquías parte/todo**

Consecuencias

- Jerarquía con clases que modelan objetos primitivos y objetos compuestos, que permite composición recursiva.
- Es fácil añadir nuevos tipos de componentes.
- No se puede confiar al sistema de tipos que asegure que un objeto compuesto sólo contendrá objetos de ciertas clases, necesidad de comprobaciones en tiempo de ejecución.

Implementación

- Referencias de componentes hijos a su padre pueden ayudar al recorrido y manejo de la estructura compuesta.
- (GoF) Operaciones de añadir, eliminar y recuperar hijos pueden estar en `Componente` o `Composite`.
- La estructura de datos adecuada para representar un composite dependerá del contexto

Decorator (Decorador)

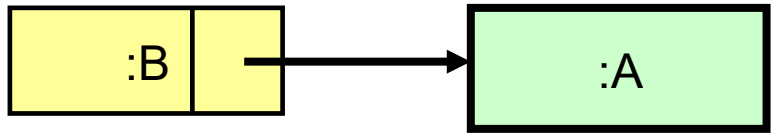
Propósito:

Asignar dinámicamente nuevas responsabilidades a un objeto. Alternativa más flexible a crear subclases para extender la funcionalidad de una clase.

Motivación

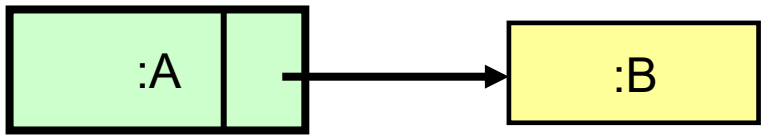
Algunas veces se desea añadir atributos o comportamiento adicional a un objeto concreto no a una clase y herencia no lo permite, por ejemplo: bordes o *scrolling* a una ventana.

¿Cómo extender dinámicamente la funcionalidad de una instancia de una clase B?



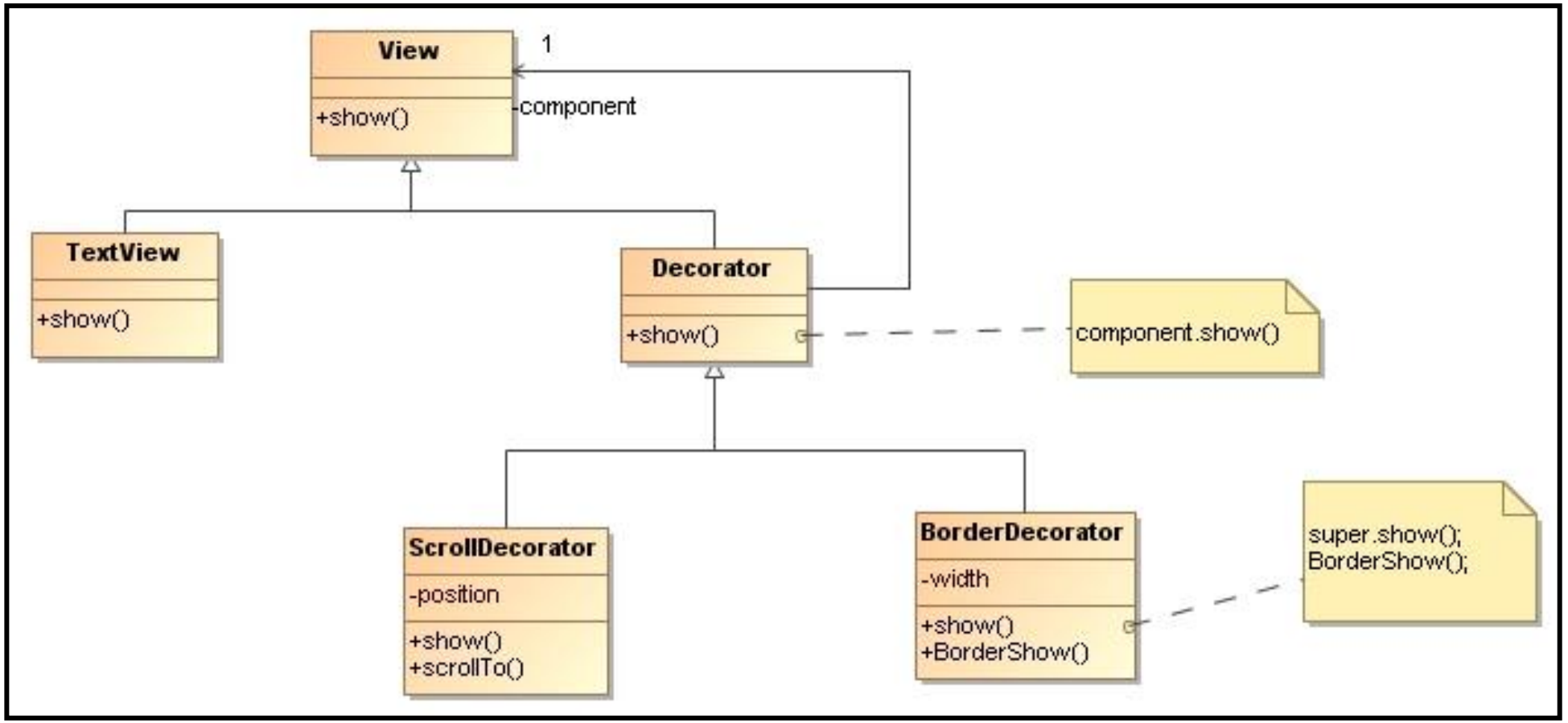
Conectar el objeto a otro con la nueva funcionalidad **(patrón Estrategia)**

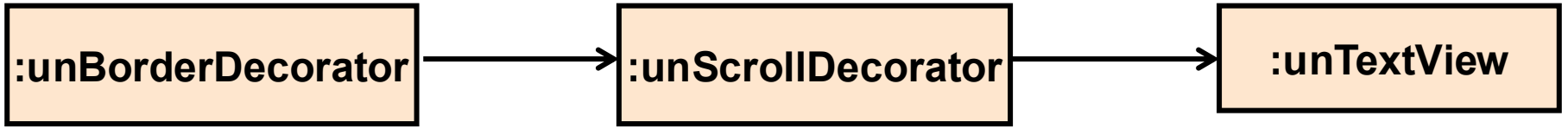
y siempre que espero un B debería poder recibir un objeto de la clase A



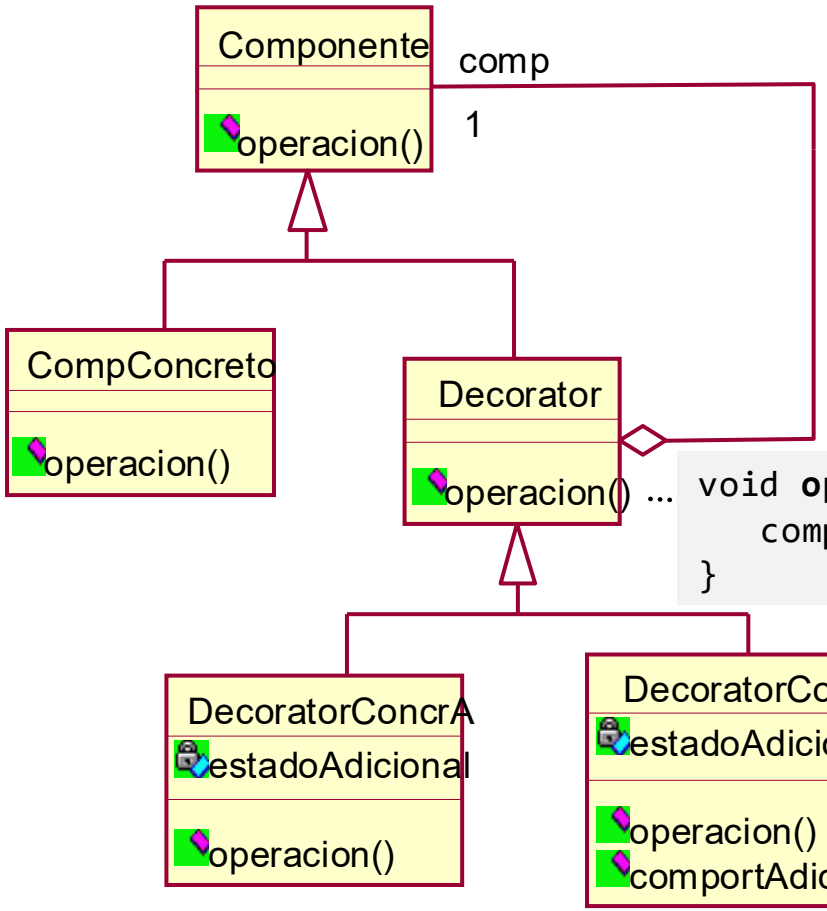
"Decorar" el objeto con otro con la nueva funcionalidad **(patrón Decorador)**

Decorador: Ejemplo ventanas con scroll y bordes





Decorador: Estructura




```
void operacion(){
   comp.operacion();
}
```

```
void operacion(){
   super.operacion();
   comportAdicional();
}
```

Decorador: Consecuencias

- Más flexible que la herencia: responsabilidades pueden añadirse y eliminarse en tiempo de ejecución.
- Diferentes decoradores pueden ser conectados a un mismo objeto.

Decorador: Implementación

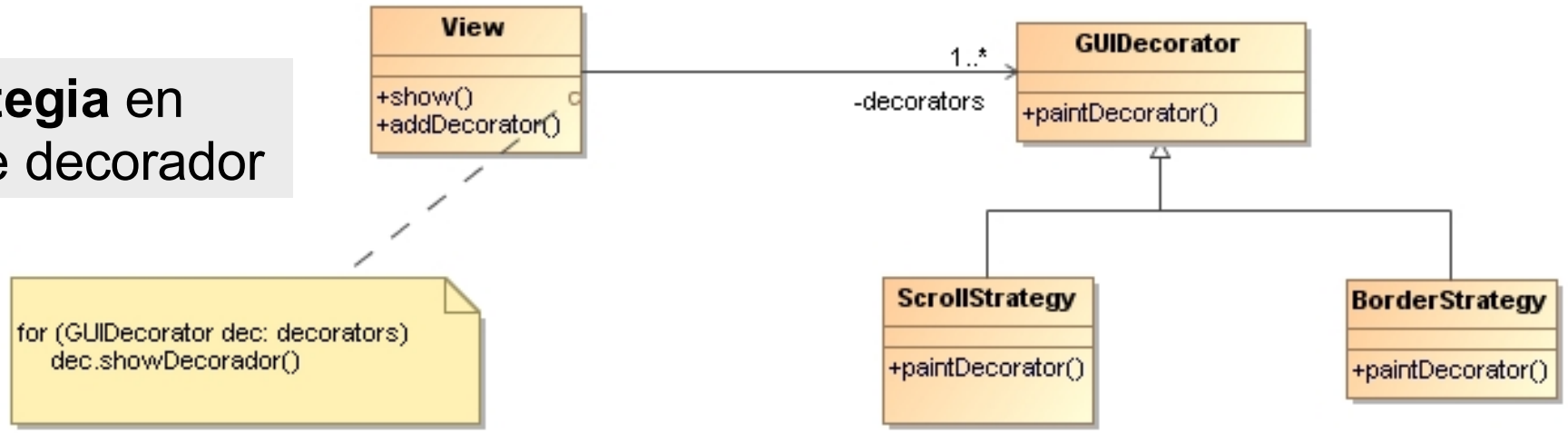
- Componentes y decoradores deben heredar de una clase común que debe ser “ligera” en funcionalidad.
- Si no es ligera es mejor usar el patrón Estrategia.

Decorador vs. Estrategia

Un componente no sabe nada acerca de sus decoradores, con *Estrategia* sucede lo contrario.

El decorador envía los mensajes al componente que decora, pudiendo extender la operación con nuevo comportamiento.

Estrategia en vez de decorador

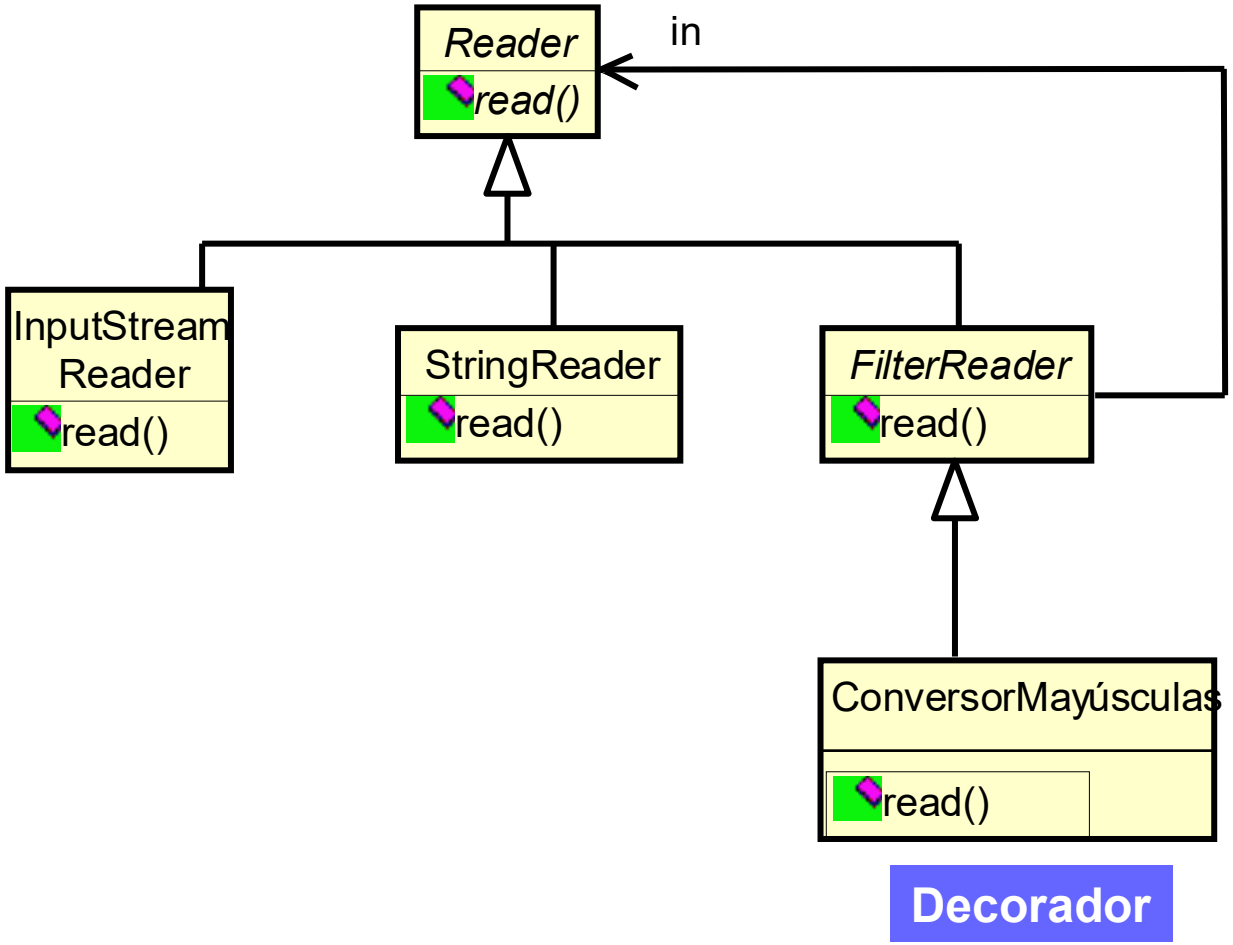


Decorador: Ejemplo *filtros de streams (entrada/salida) en Java*

Java posee una librería de clases e interfaces para manejar streams de caracteres y bytes. Por ejemplo, la clase abstracta `Reader` que proporciona un stream de caracteres tiene subclases tales como `InputStreamReader` o `StringReader` que implementan diferentes formas de proporcionar un stream de entrada de caracteres. También se dispone de una clase `Writer` para streams de salida de caracteres. Para cada uno de los tipos de stream, la librería incluye unas clases que representan "filtros de stream" (*filter*) que permiten encadenar filtros sobre un stream y aplicarle varias transformaciones, como son las clases abstractas `FilterReader` y `FilterWriter`. A continuación, se muestra parte del código de la clase `FilterReader` que implementa métodos de la clase `Reader` como `read()` y que es raíz de clases que implementan filtros de lectura.

```
public abstract class FilterReader
                              extends Reader {..
  protected Reader in;
  protected FilterReader(Reader in){
        super(in);
        this.in = in;
  }
  public int read() throws IOException {
        return in.read();
  }
}
```

Decorador: Ejemplo "*Filtros de streams en Java*"




```
public class ConversorMayúsculas
                extends FilterReader {

   public ConversorMayúsculas (Reader in) {
                        super(in);
    }

    public int read() throws IOException {
        int c = super.read();
                if (c != -1)
                   return Character
                          .toUpperCase((char)c);
                else return c;
    }
}
```

Facade (Fachada)

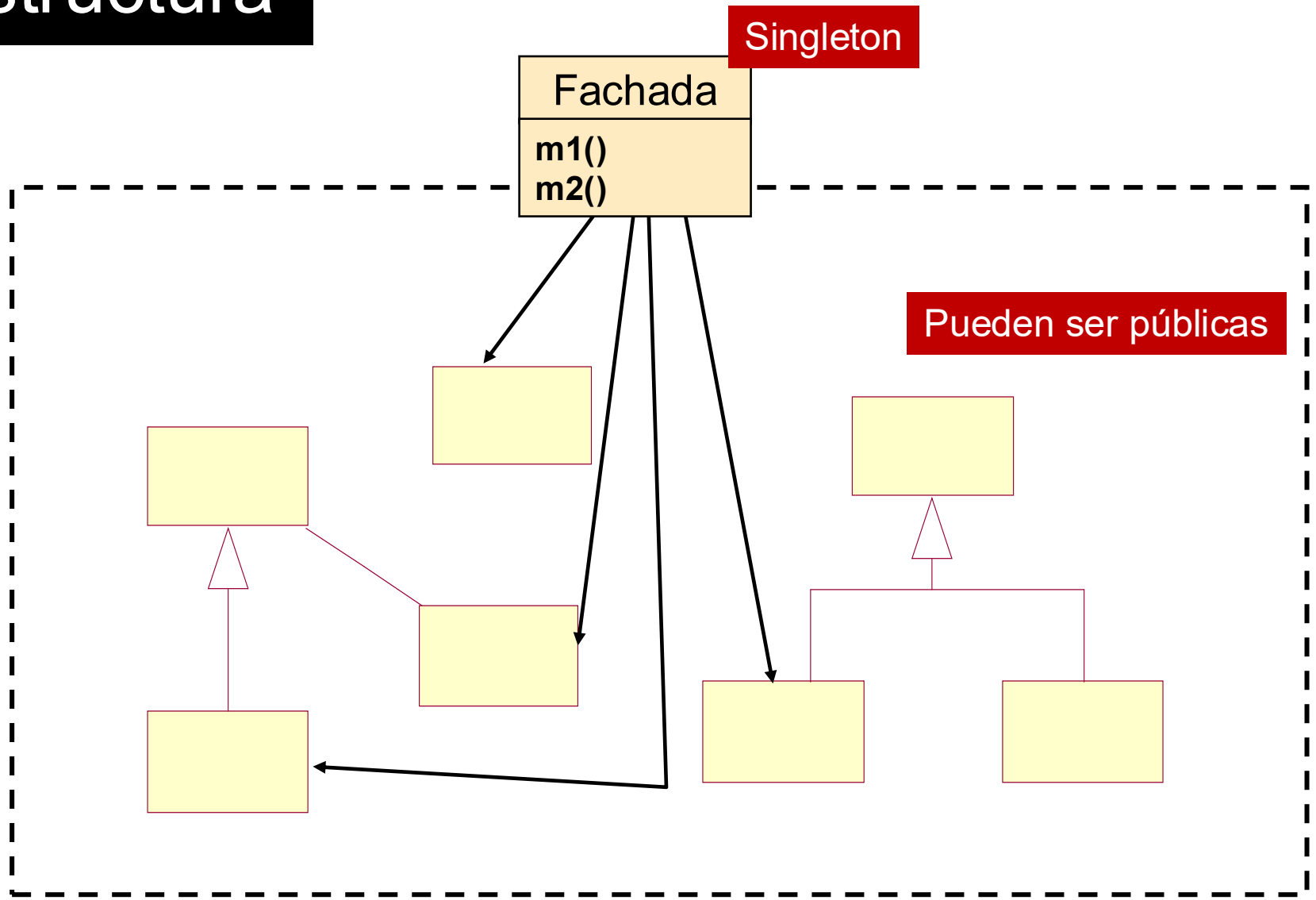
Propósito

- Proporciona una única interfaz a un conjunto de clases de un subsistema que ofrece una funcionalidad. Define una interfaz de más alto nivel que facilita el uso de un subsistema.

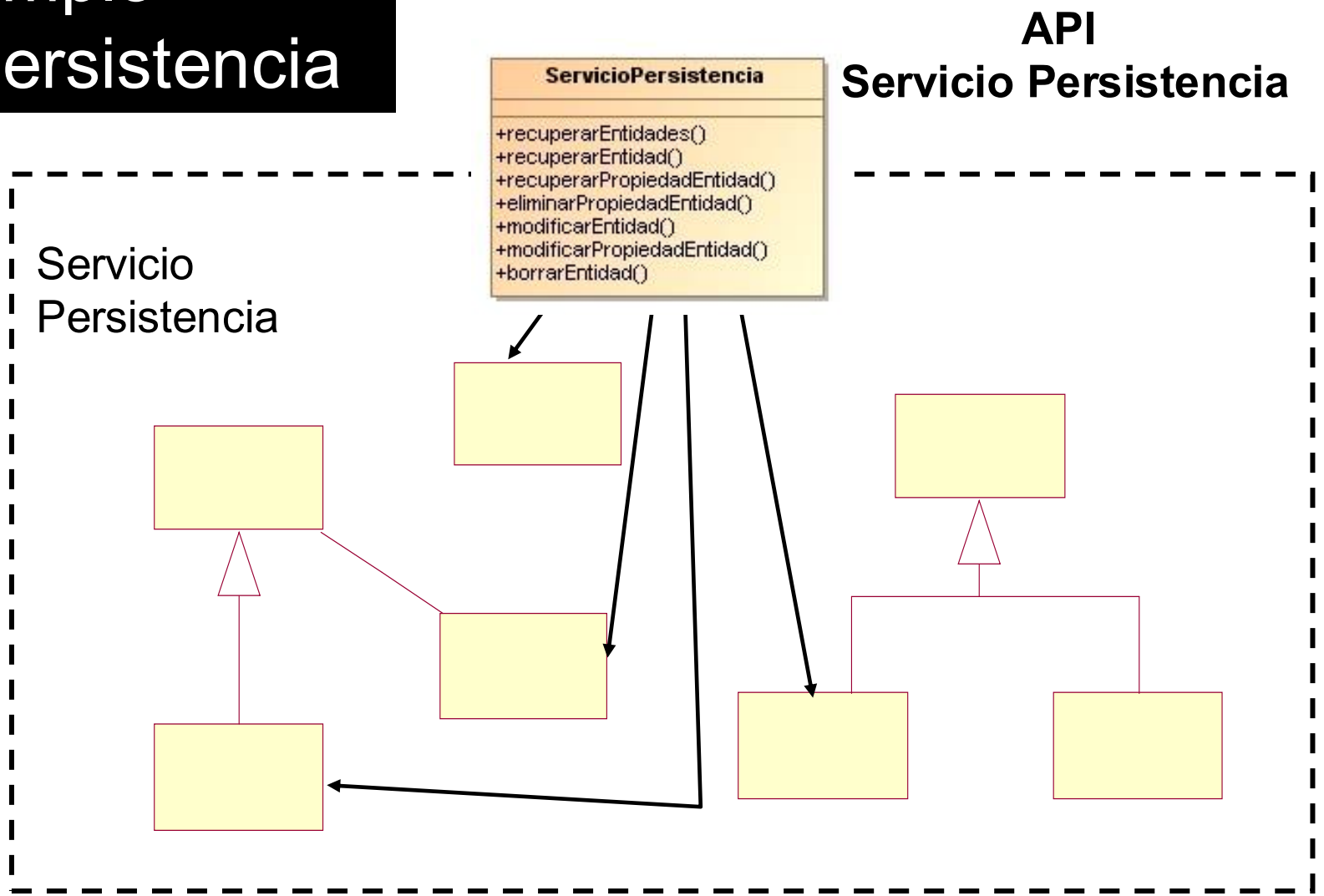
Motivación

- Reducir las dependencias (acoplamiento) entre subsistemas.
- Servicio Persistencia utilizado en el caso práctico de la asignatura
- Controlador entre la capa GUI y la capa de negocio.

Fachada:Estructura



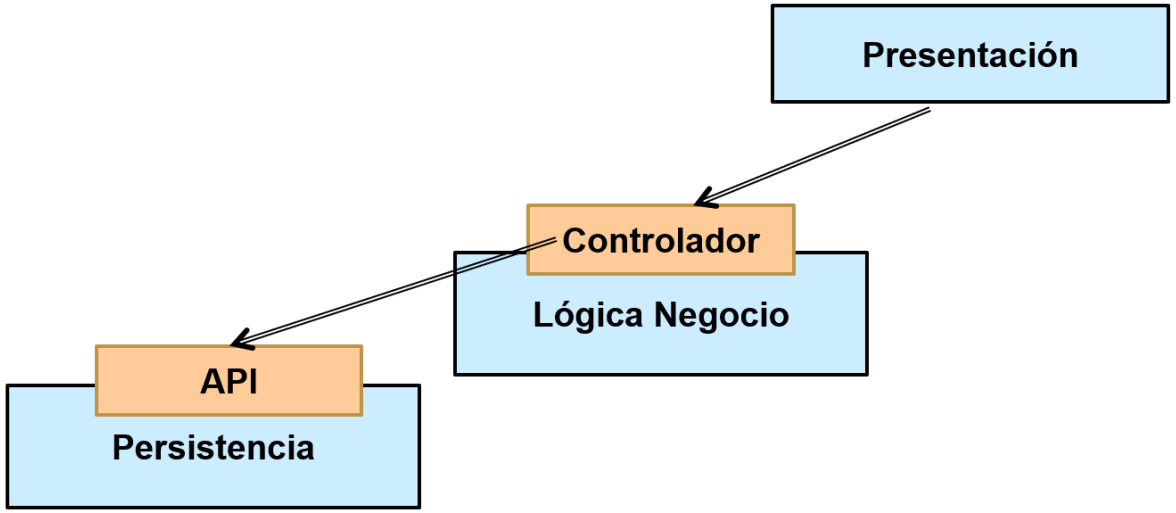
Fachada: Ejemplo Servicio de Persistencia




```
ServicioPersistencia servPersistencia =
        FactoriaServicioPersistencia.INSTANCE().
                                getServicioPersistencia();
```

Fachada: Aplicabilidad

- Proporcionar una interfaz simple a un subsistema.
- Se desea una arquitectura de varios niveles: una fachada define el punto de entrada para cada nivel-subsistema.



Fachada: Consecuencia

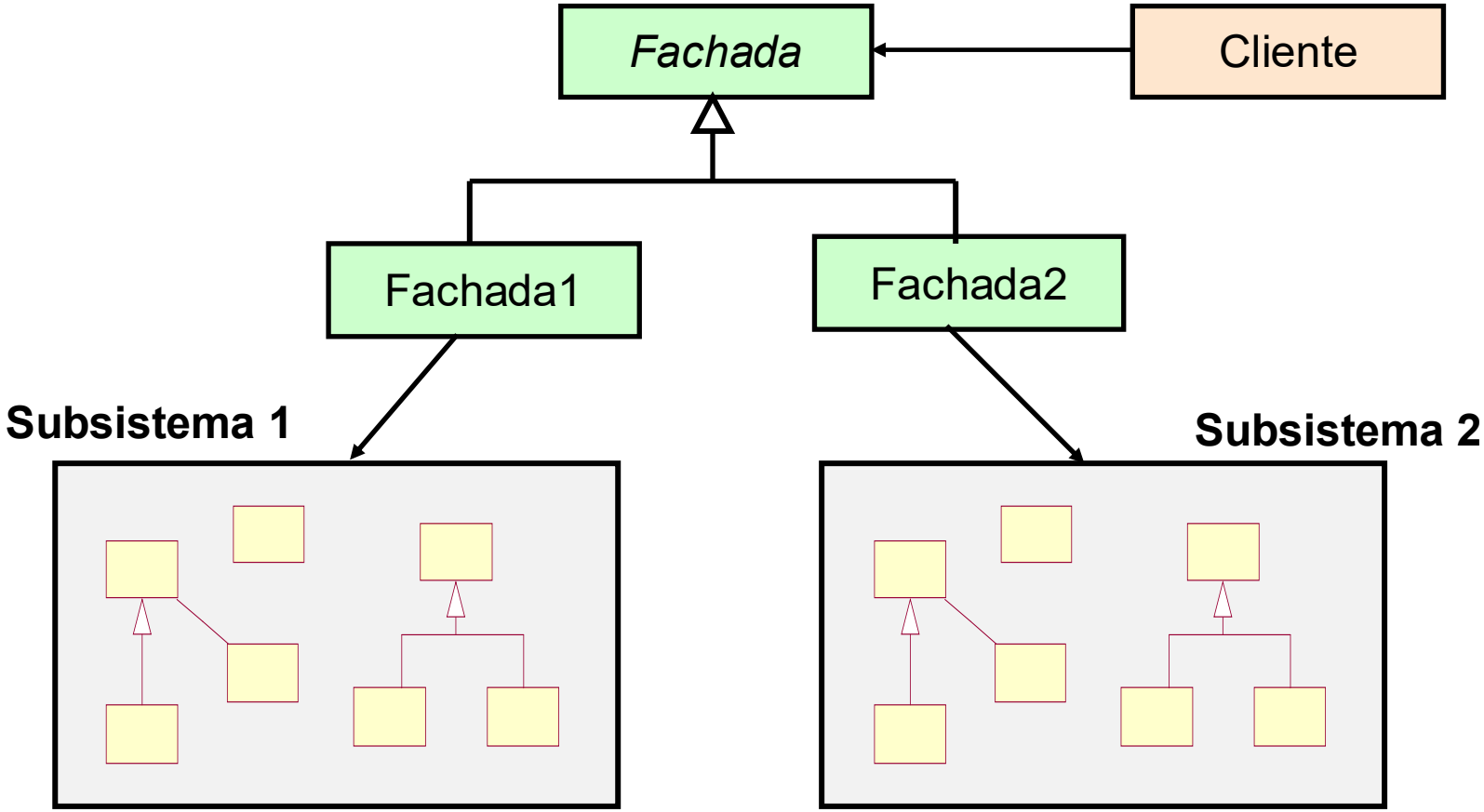
- **Facilita el uso** de un subsistema al ocultar sus componentes.
- Proporciona un **acoplamiento débil** entre un subsistema y los clientes: cambios en los componentes no afectan a los clientes.
- **No se impide a los clientes** el uso de las clases del subsistema si lo necesitan.

Fachada: Implementación

- Una fachada es normalmente un **Singleton**.
- Es posible declarar clases individuales del subsistema como públicas.
- **Caso de un sistema con varias implementaciones**: Fachada es una clase abstracta con una subclase por cada implementación del subsistema.

Fachada: Jerarquía de fachadas

Diferentes implementaciones del sistema (p.e., una librería)



¿Diferencia con adaptador?